实验二　8 位移位相加乘法器 (Wallace 树)

1 实验背景

乘法器用于计算两个数的乘积，是数字滤波、相关计算、点积和矩阵运算中的基本算术模块。神经网络和数字信号处理中的大量乘加操作，都建立在整数乘法器和加法器之上。

二进制乘法可以展开为多个移位部分积之和。如果逐个使用普通进位传播加法器串行累加，每一级都会产生完整的进位传播，组合关键路径会随部分积数量增加而增长。

Wallace 树的核心思想不是立即求出每一步的完整二进制和，而是使用进位保存加法器（Carry-Save Adder，CSA）不断把多个部分积压缩成更少的向量。压缩过程中不进行全字长横向进位传播，直到最后只剩两个待加向量，再使用快速加法器完成最终求和。

本实验以 8 位无符号整数为对象，把“部分积生成—Wallace 压缩—并行前缀最终加法”连接起来，并通过与朴素连加结构比较，理解面积与逻辑深度之间的 tradeoff。

2. 实验原理

2.1 实验原理

（1）从二进制展开得到部分积。

令 A、B 为 8 位无符号整数，B 的第 j 位为 b_j，则乘积可写为：

$$\text{res} = \Sigma_j \left[b_j \ ? \ (A \ll j) \ : \ 0 \right], \qquad j = 0 \cdots 7$$

因此可以并行生成 8 个 16 位部分积 pp[0]…pp[7]。当 b_j=0 时该行全 0；当 b_j=1 时该行等于 A 左移 j 位后的值。移位前应先扩展到足够宽度，避免中间表达式提前截断。

（2）用进位保存加法器压缩多个操作数。

CSA 是一个 3:2 压缩器：输入三个等宽向量 x、y、z，输出和向量 s 与进位向量 q。对每一位分别计算：

$$s = x \oplus y \oplus z$$

$$q = xy + xz + yz$$

$$x + y + z = s + (q \ll 1)$$

关键在于每一位可以独立计算，不需要等待相邻低位的横向进位。因此一个 CSA 层的逻辑深度不会随着向量位宽线性增加。实现时必须统一约定 q 是否已经左移；本参考实现输出未左移的 q，并在进入下一级时统一左移 1 位。

（3）Wallace 树逐层压缩，最后完成进位传播。

采用向量级 CSA 组织压缩树时，8 个部分积按照 8→6→4→3→2 的方式逐层减少，不足三个的向量直接传递到下一层。

层	输入向量数	CSA 组织方式	输出向量数
初始	8（pp0…pp7）	—	8
第 1 层	8	2 个 CSA，各压缩 3 个；其余 2 个直通	6
第 2 层	6	2 个 CSA	4
第 3 层	4	1 个 CSA；1 个向量直通	3
第 4 层	3	1 个 CSA	2

最后两个向量需要执行一次真正的进位传播加法。本实验要求复用实验一的 19 位并行前缀进位加法器：将两个 16 位向量零扩展到 19 位，cin=0，取 sum[15:0] 作为最终乘积。

8 位无符号乘法的最大值为 255×255=65025，小于 2^{16}，因此 16 位能够完整表示结果；接入 19 位 ADDER 是为了模块复用，正确结果的更高位应保持为 0。3. 实验内容和要求

使用 Verilog 实现以下模块，并结合实验原理完成设计。

2.2 模块接口

模块名称：MULT

信号	方向	说明
in1[7:0]、in2[7:0]	输入	两个 8 位无符号整数
res[15:0]	输出	16 位乘积

2.3 实现要求

1. 完成 8 位无符号整数的移位相加乘法器，采用组合逻辑实现。

2. 8 个部分积的相加采用 Wallace Tree 结构进行压缩。

3. Wallace 树压缩得到的两个部分和，使用实验一设计的并行前缀进位加法器完成最终求和。

4. 实验报告应给出部分积、CSA、压缩过程、模块结构、验证方法以及面积与最长拓扑路径的对比分析。

3 参考实现与代码阅读

文件	作用
lab_2/rtl/CSA.v	参数化向量级 CSA；s、q 中 q 为未左移的原始进位
lab_2/rtl/MULT8_WALLACE.v	部分积生成 → 4 层 CSA → Lab1 ADDER 最终求和
lab_2/rtl/MULT.v	实验要求的顶层包装，模块名 MULT

文件	作用
lab_2/rtl/MULT_NAIVE.v	朴素连加对照设计

1. **部分积生成：**pp[j] = in2[j] ? ({8'b0, in1} << j) : 0，先扩展到 16 位再移位。

2. **进位左移：**CSA 输出的 q 在送入下一层时左移一位，例如 {q[W-2:0], 1'b0}。忘记移位或重复移位都会产生系统性错误。

3. **直通向量：**每一层不足三个的剩余向量必须完整传到下一层，不能遗漏或重复。

4. **最终加法：**两个 16 位压缩结果零扩展到 19 位后送入实验一 ADDER，cin=0，取低 16 位作为 res。

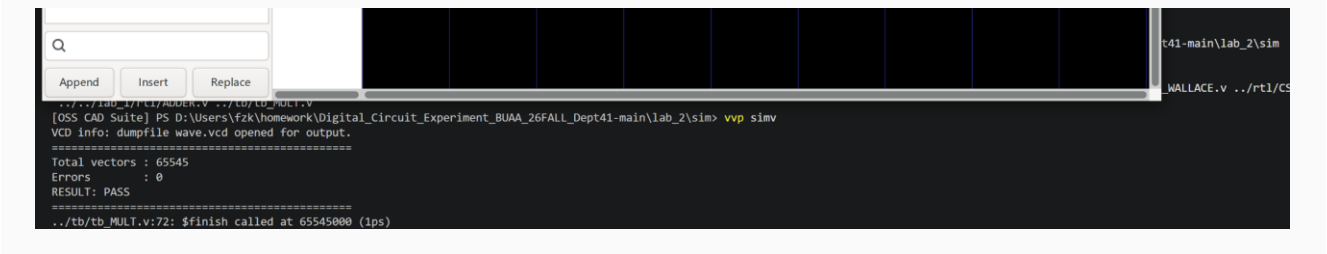
4 验证方案与结果

测试文件：lab_2/tb/tb_MULT.v。验证采用定向用例加全量遍历。8 位无符号乘法器只有 256×256=65536 种输入组合，因此可以直接穷举全部输入，并与行为级参考 in1*in2 逐一比对。

类别	内容	目的
定向 9 组	0×0、0×N、1×N、255×255、255×1、128×2、幂次等	覆盖零值、边界、进位与移位
全量 65536 组	a=0…255，b=0…255，与 a*b 逐一比较	穷举全部输入空间

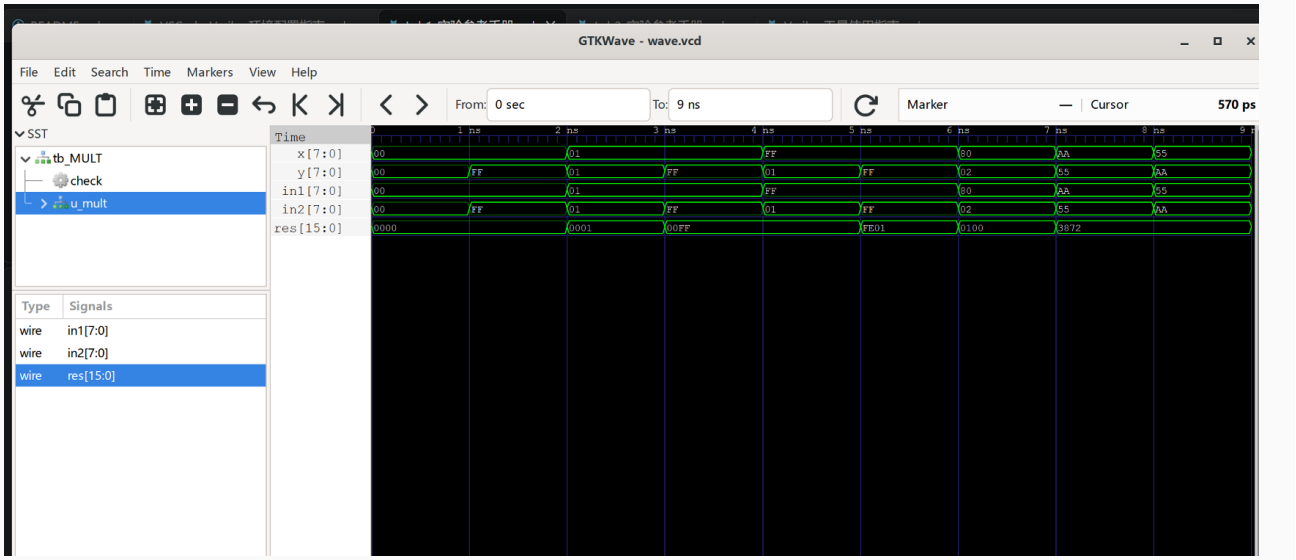
```
cd lab_2/sim
iverilog -g2005 -o
simv ../rtl/MULT.v ../rtl/MULT8_WALLACE.v ../rtl/MULT_NAIVE.v ../rtl/CSA.v ../../lab_1/rtl/ADDER.v ../tb/t
b_MULT.v
vvp simv
```

本次重新实测中，Wallace 顶层 MULT 完成 9 组定向测试，并对 256×256=65536 种输入组合进行完整穷举；终端共执行 65545 次 check，Errors: 0，最终输出 RESULT: PASS。MULT_NAIVE 作为后续综合性能对照设计，当前 tb_MULT 未对其进行穷举验证。



实验二 Testbench 仿真结果：全部输入空间穷举通过

结果说明：终端显示 Total vectors = 65545、Errors = 0、RESULT: PASS。其中 65545 次 check 由 9 组定向测试与 65536 组穷举测试组成；65536 组穷举正好覆盖两个 8 位无符号输入的全部 256×256 种组合。因此当前 Wallace 顶层 MULT 在完整输入空间内均与行为级参考 x*y 一致。



实验二 9 组定向测试的 GTKWave 波形

波形说明：VCD 仅记录前 9 组定向测试，因此图中时间范围为 0～9 ns，每 1 ns 对应一组输入。波形中可直接看到若干关键边界结果：0xFF×0xFF=0xFE01（65025）、0x80×0x02=0x0100（256）、0xAA×0x55=0x3872，且交换操作数后结果仍为 0x3872。完整的 65536 组穷举在 $dumpoff 之后继续执行，所以不会全部写入 wave.vcd。

5 综合结果与分析

```
cd lab_2/syn && yosys synth.ys
```

设计	cells	最长拓扑路径	理论结构
Wallace Tree	381	28 级	4 层 CSA + 前缀最终加法
朴素连加	338	37 级	7 次普通加法串接

参考综合结果表明，Wallace Tree 相比朴素连加增加约 13% 的 cells，但 longest topological path 从 37 级降低到 28 级，缩短约 24%。这体现了典型的面积换逻辑深度。

需要注意，理论上的“4 层 CSA + 1 次最终加法”不会原样保留在门级网表中。Yosys/ABC 会对逻辑进行全局重构和门级映射，因此 28 与 37 是特定综合流程下的拓扑深度指标，不是简单把理论层数相加得到的值，也不等同于实际工艺中的纳秒延迟。

为了保证对比公平，Wallace 与朴素设计应采用相同的综合脚本、相同的 top、相同的 flatten 流程和相同的映射条件。本参考脚本使用 synth -top MULT -flatten，使综合器可以在整平后的设计上进行全局优化。

```
[OSS CAD Suite] PS D:\Users\fzk\homework\Digital_Circuit_Experiment_BUAA_26FALL_Dept41-main\lab_2\syn> Write-Host "===== Wallace Tree ====="
>> Get-Content .\wallace_stat.txt | Select-String '=== MULT ===','^\s*\d+\s+cells\s*$'
>> Get-Content .\wallace_ltp.txt  | Select-String '^Longest topological path'
>>
>> Write-Host ""
>> Write-Host "===== Naive Multiplier ====="
>> Get-Content .\naive_stat.txt | Select-String '=== MULT_NAIVE ===','^\s*\d+\s+cells\s*$'
>> Get-Content .\naive_ltp.txt  | Select-String '^Longest topological path'
===== Wallace Tree =====

=== MULT ===
      381 cells
Longest topological path in MULT (length=28):

===== Naive Multiplier =====
=== MULT_NAIVE ===
      338 cells
Longest topological path in MULT_NAIVE (length=37):
```

实验二 Yosys 综合结果对比

结果说明：Wallace Tree 乘法器综合后为 381 cells，最长拓扑路径为 28 级；朴素移位连加乘法器为 338 cells，最长拓扑路径为 37 级。Wallace 结构的 cells 增加约 13%，但 longest topological path 缩短约 24%。这说明 Wallace Tree 通过额外的 CSA 压缩逻辑减少了多操作数求和中的串行进位依赖，体现了典型的面积—逻辑深度权衡。28/37 同样是特定 Yosys/ABC 综合流程下的拓扑级数，不应直接解释为实际传播延迟。

6 实验报告写作建议

- **原理部分：**绘制 8 行错位部分积、CSA 3:2 压缩关系以及 8→6→4→3→2 的压缩过程。
- **结构部分：**画出 MULT → MULT8_WALLACE → CSA + ADDER 的模块层次，说明为什么最终必须进行一次真正的进位传播加法。
- **验证部分：**强调 8 位输入空间可穷举，给出 65536 组全部通过的结果，并附边界用例波形。
- **综合对比：**使用自己的 Wallace 与对照实现，在同一综合流程下比较 cells 和 longest topological path。
- **tradeoff 理解：**结合数据说明 Wallace 并非“门数最少”，其优势主要在于减少多个部分积求和过程中的串行进位依赖。
- **改进方向：**可进一步讨论 Booth 编码减少部分积数量、4-2 压缩器或流水线化等方法。

7 常见问题与课程衔接

问题	原因 / 建议
结果总是差一倍或 2 倍	重点检查 CSA 的 q 是否恰好左移一次
大数相乘错误、小数正确	检查部分积是否先扩展到 16 位，以及移位后是否被截断
Unknown module: ADDER	编译时遗漏 ../../lab_1/rtl/ADDER.v
综合 LTP 明显异常	检查两个设计是否使用相同的 -top、-flatten 和综合流程
全量遍历运行过慢	Testbench 应只在错误时打印，不应对 65536 组逐条 $display

本实验位于课程“加法器→乘法器→矩阵计算阵列”的中间环节：最终求和直接复用实验一 ADDER；在实验三复用版中，MULT8_WALLACE 可继续作为处理单元中的乘法模块，再与实验一的加法器组成乘加数据通路。